

Exercice GRASP - Atelier de réparation de vélo

Ces exercices sont à adapter à votre propre schéma de classe d'analyse. Ils vous permettront en même temps de constater que votre schéma est cohérent pour une mise en oeuvre

Exercice 1 : Expert de l'information (Information Expert)

Le principe : Attribuer une responsabilité à la classe qui détient l'information nécessaire pour la remplir. C'est le principe de base pour l'attribution des responsabilités.

Scénario : Un client a accepté un devis. Nous devons calculer le montant total de ce devis.

Exercice :

1. Où placer la responsabilité (la méthode) `calculerMontantTotal()` ?
 2. Quelles informations sont nécessaires pour ce calcul ?
 3. Quelles classes détiennent ces informations ?
 4. Faire le diagramme de classes avec la méthode ajoutée.
 5. Faire le diagramme de séquence de l'appel à cette méthode.
-

Exercice 2 : Créateur (Creator)

Le principe : Qui doit être responsable de la création d'une instance d'un objet `A` ? Une classe `B` qui :

- Contient ou agrège `A`.
- Enregistre `A`.
- Utilise étroitement `A`.
- Possède les données d'initialisation pour `A`.

Scénario : Un mécanicien, lors d'un rendez-vous, établit un devis pour le client.

Exercice :

1. Quelle classe doit être responsable de la création de l'objet `Devis` ?
 2. Quelle classe doit être responsable de la création des objets `LigneDevis` ?
 3. Faites le diagramme de classes avec les méthodes de création ajoutées.
 4. Faites le diagramme de séquence de la création d'un devis avec plusieurs lignes.
-

Exercice 3 : Contrôleur (Controller)

Le principe : Attribuer la responsabilité de la gestion d'un événement système (par ex. un clic d'utilisateur) à une classe qui n'est pas l'interface utilisateur (UI) elle-même. Cette classe représente :

1. Le système global (Contrôleur de Façade).
2. Un scénario ou cas d'utilisation (Contrôleur de Cas d'Utilisation).

Scénario : Un client, depuis l'application web, soumet le formulaire "Prendre un nouveau rendez-vous".

Exercice :

1. L'interface (la page web) doit-elle créer elle-même l'objet `RendezVous` ?
 2. La classe `Client` doit-elle recevoir la requête HTTP ?
 3. Quelle classe devrait recevoir et coordonner cette demande ?
 4. Faites le diagramme de classes avec le contrôleur ajouté.
 5. Faites le diagramme de séquence de la prise de rendez-vous.
-

Exercice 4 : Forte Cohésion (High Cohesion)

Le principe : Maintenir les responsabilités d'une classe ciblées, gérables et étroitement liées. Une classe ne doit pas "tout faire".

Scénario : La classe `RendezVous` a de nombreuses responsabilités :

- Connaître les détails du RDV (Date, Etat, Notes...).
- Être liée au `Client` et au `Velo`.
- Générer un `Devis` (vu dans Créateur).
- Gérer l'état du paiement (`PaieementEnLigne`).

Imaginons maintenant qu'on veuille y ajouter la responsabilité de **gérer le stock** (c-à-d, décrémenter le stock des pièces utilisées lors du RDV).

Exercice :

1. Est-ce une bonne idée d'ajouter la méthode `decrementerStockPieces()` à la classe `RendezVous` ?
 2. Qu'arriverait-il à la cohésion de `RendezVous` ?
-

Exercice 5 : Fabrication Pure (Pure Fabrication)

Le principe : Créer une classe "artificielle" (qui n'existe pas dans le domaine métier) pour préserver la Forte Cohésion et le Faible Couplage.

Scénario : (Suite de l'exercice 4) Nous devons gérer la logique de stock (réserver des pièces, décrémenter les quantités). Nous avons refusé de la mettre dans `RendezVous` pour garder une Forte Cohésion.

Exercice :

1. `Stock` est une classe de données. Doit-elle contenir la *logique* complexe de réservation ?
2. Où placer cette responsabilité ?
3. Faites le diagramme de classes avec la nouvelle classe ajoutée.
4. Faites le diagramme de séquence de la réservation de pièces lors de la confirmation d'un rendez-vous.

Exercice 6 : Faible Couplage (Low Coupling)

Le principe : Minimiser les dépendances entre les classes. Un changement dans une classe A ne doit pas nécessiter de changements dans une classe B.

Scénario : Le `RendezVous` doit être payé (`PaieementEnLigne`). Imaginons que nous utilisons une API externe, comme "Stripe".

Exercice :

1. La classe `RendezVous` doit-elle contenir du code qui appelle directement `Stripe.creerPaielement(...)` ?
 2. Que se passe-t-il si l'on décide de changer de prestataire pour "PayPal" ?
 3. Faites le diagramme de classes avec la nouvelle conception.
 4. Faites le diagramme de séquence de l'appel au paiement en ligne.
-

Exercice 7 : Polymorphisme (Polymorphism)

Le principe : Quand un comportement varie selon le type, utiliser des opérations polymorphes (méthodes sur une interface/super-classe) plutôt que des `if/else` ou `switch` basés sur le type.

Scénario : L'atelier répare différents types de vélos. La réparation d'un `VeloElectrique` est différente de celle d'un `VeloDeRoute` (ex: diagnostic batterie vs réglage dérailleur).

Exercice :

1. Dans la méthode `effectuerReparation()` du `Mecanicien`, doit-on écrire : `if (velo.Type == "Electrique") { ... } else if (velo.Type == "Route") { ... } ?`
 2. Comment améliorer cette conception ?
 3. Faites le diagramme de classes avec la nouvelle conception.
-

Exercices 8 & 9 : Variations Protégées (Protected Variations) & Indirection (Indirection)

Ces deux principes sont souvent utilisés ensemble (comme dans l'exercice 6).

- **Indirection :** Attribuer la responsabilité à un objet intermédiaire pour découpler deux composants (ex: `GestionnaireStock`, `IServicePaiement`).
- **Variations Protégées :** Identifier les points d'instabilité (ce qui va changer) et créer une interface stable autour d'eux.

Scénario : Le système doit notifier le client lorsque son vélo est prêt. Aujourd'hui, on envoie un Email. Demain, on veut pouvoir envoyer des SMS ou des notifications Push.

Exercice :

1. Où placer la logique d'envoi de l'email ? Dans `RendezVous` ? Dans `Client` ?
2. Comment s'assurer qu'on pourra ajouter le SMS sans modifier les classes `RendezVous` ou `Client` ?
3. Faites le diagramme de classes avec la nouvelle conception.
4. Faites le diagramme de séquence de la notification.

Gestion de la persistance

Exercice 1 : Le problème de la Forte Cohésion (High Cohesion)

Le principe : Une classe doit avoir un ensemble de responsabilités ciblées et étroitement liées. Mélanger la logique métier avec la logique technique (comme le SQL) brise la cohésion.

Scénario : Un nouvel utilisateur s'inscrit. L'application doit créer un nouvel objet **Client** et le sauvegarder dans la base de données.

Exercice :

1. Un étudiant propose d'ajouter une méthode **sauvegarderEnBaseDeDonnees()** directement à la classe **Client**.
 2. Pourquoi est-ce une **mauvaise idée** ? Quel(s) principe(s) GRASP cela viole-t-il ?
 3. Quelles seraient les conséquences si on changeait de base de données (passer de MySQL à PostgreSQL, ou à un NoSQL comme MongoDB) ?
-

Exercice 2 : La solution par Fabrication Pure (Pure Fabrication)

Le principe : Inventer une classe "artificielle" (qui n'existe pas dans le domaine métier) pour préserver la Forte Cohésion et le Faible Couplage des objets du domaine. C'est la solution directe à l'exercice 1.

Scénario : Nous avons établi qu'on ne peut pas mettre la logique de sauvegarde dans **Client**. Où la mettre ?

Exercice :

1. En appliquant le principe de **Fabrication Pure**, quelle(s) nouvelle(s) classe(s) devrions-nous inventer pour gérer la persistance des objets **Client** ?
 2. Quelles seraient les responsabilités (méthodes) de cette nouvelle classe ?
 3. Faites le diagramme de classes avec la nouvelle classe ajoutée.
 4. Faites le diagramme de séquence de la création et sauvegarde d'un nouvel objet **Client**.
-

Exercice 3 : Le flux complet (Contrôleur + Indirection)

Les principes :

- **Contrôleur** : Gère les événements système.
- **Indirection** : Utiliser un objet intermédiaire pour découpler des composants. Le Repository est une forme d'Indirection entre l'application et la BDD.
- **Faible Couplage** : L'objectif final.

Scénario : Reprenons le cas d'utilisation "Prendre un nouveau rendez-vous". La requête arrive à notre **RendezVousContrôleur** avec les données : **clientID**, **veloID**, **dateHeureSouhaitee**.

Exercice :

1. Le **RendezVousContrôleur** a besoin des objets **Client** et **Velo** pour créer le **RendezVous**.
 2. Comment le **Contrôleur** obtient-il ces objets ?
 3. Une fois l'objet **RendezVous** créé (par **RendezVous = new RendezVous(...)**), comment est-il sauvegardé ?
 4. Dessinez le diagramme de séquence simplifié de ce flux.
-

Exercice 4 : Variations Protégées (Protected Variations)

Le principe : Identifier les points d'instabilité (variations) et créer une interface stable autour d'eux pour protéger le reste du système.

Scénario : Notre **ClientRepository** de l'exercice 2 fonctionne très bien avec une base de données MySQL. L'entreprise envisage de créer une version "hors-ligne" de l'application qui sauvegarderait les données dans des fichiers XML locaux.

Exercice :

1. La "manière de sauvegarder" (MySQL vs XML) est un **point de variation**.
2. Le **RendezVousContrôleur** (Exercice 3) dépend de **ClientRepository**. Si on crée **ClientXMLRepository**, faut-il modifier le **Contrôleur** ?
3. Comment appliquer **Variations Protégées** pour que le **Contrôleur** n'ait pas besoin de savoir *comment* les clients sont sauvegardés ?
4. Faites le diagramme de classes avec la nouvelle conception.